

EXAMEN FINAL

Nombre: Cristhyan Manuel Perez Villegas

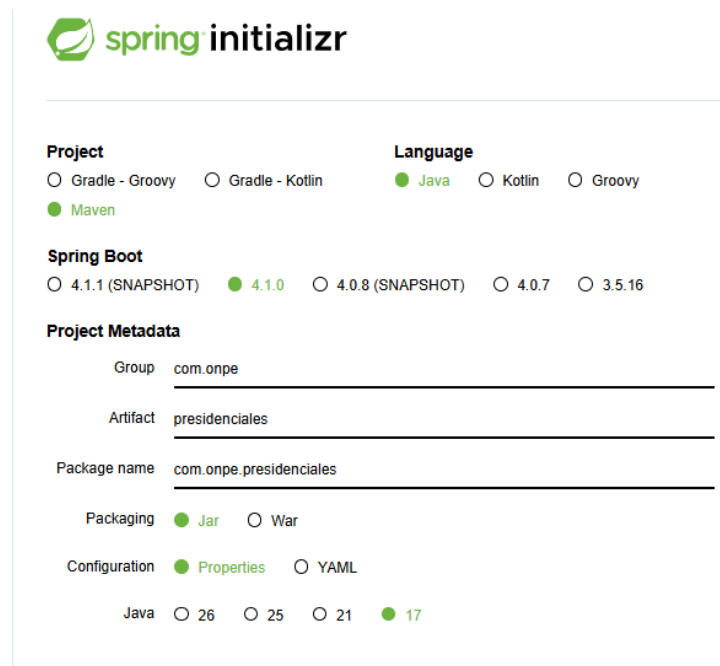
Código: 20231066A

Caso: ONPE - Elección de Fórmula Presidencial (2° vuelta)

1.-Para la HU: Resultados de la Elección de Fórmula Presidencial: Implementar lo siguiente:

a) Implementación de apis a nivel backend (3.5pts)

Bien aquí empezamos con “Spring Initializr”;



The image shows the Spring Initializr web form. It has a green logo and the text 'spring initializr'. Below the logo, there are several sections with radio buttons for selection:

- Project**: ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Maven
- Language**: ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot**: ☐ 4.1.1 (SNAPSHOT), ☒ 4.1.0, ☐ 4.0.8 (SNAPSHOT), ☐ 4.0.7, ☐ 3.5.16
- Project Metadata**:
 - Group:
 - Artifact:
 - Package name:
 - Packaging: ☒ Jar, ☐ War
 - Configuration: ☒ Properties, ☐ YAML
 - Java: ☐ 26, ☐ 25, ☐ 21, ☒ 17

Y aquí configuramos las dependencias

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

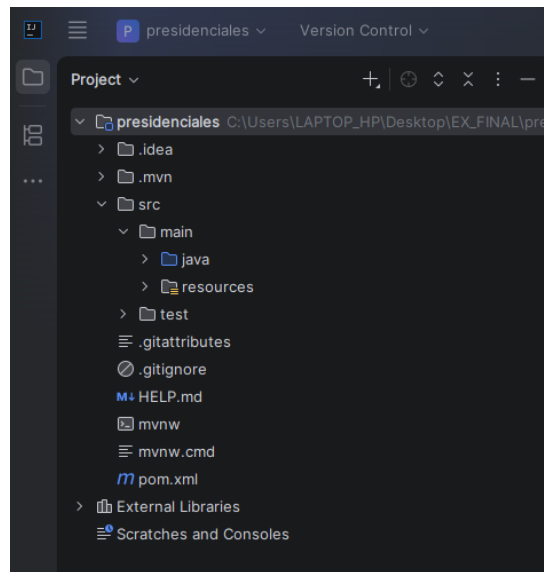
H2 Database SQL

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

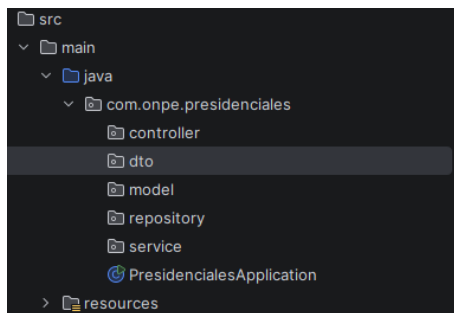
Lombok DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

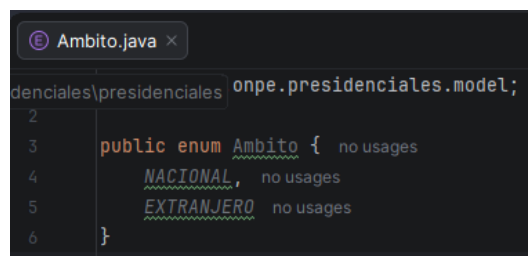
Ahora en “intelliJ IDEA” ya tenemos la estructura;



Estructuración del proyecto en capas, model que contiene las entidades; repository que contiene las interfaces de acceso a datos; service, que contiene la lógica de negocio; controller, que contiene los endpoints REST; y dto, que contiene los objetos de transferencia



Creación del enum “Ambito” en la capa model



Creamos la entidad “candidato” dentro de model;

```
E Ambito.java x C Candidato.java x
1 package com.onpe.presidenciales.model;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7 import lombok.Data;
8
9 @Entity
10 @Data
11 public class Candidato {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     private String nombre;
18     private String partido;
19 }
```

Ahora, la entidad “resultado” que va a guardar los votos de cada candidato, separados por “NACIONAL” y “EXTRANJERO”. EL “@ManyToOne” indica que muchos resultado pueden pertenecer al mismo candidato, y el “@JoinColumn(name = “candidato_id”)” indica que así se va a llamar la columna en la base de datos.

```
E Ambito.java C Candidato.java C Resultado.java x
7 import jakarta.persistence.GenerationType,
8 import jakarta.persistence.Id;
9 import jakarta.persistence.JoinColumn;
10 import jakarta.persistence.ManyToOne;
11 import lombok.Data;
12
13 @Entity
14 @Data
15 public class Resultado {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @Enumerated(EnumType.STRING)
22     private Ambito ambito;
23
24     private Long votos;
25     private Double porcentaje;
26
27     @ManyToOne
28     @JoinColumn(name = "candidato_id")
29     private Candidato candidato;
30 }
```

Ahora, configuramos el paquete “repository”.

```
Ambito.java x Candidato.java Resultado.java ResultadoRepository.java x
1 package com.onpe.presidenciales.repository;
2
3 import com.onpe.presidenciales.model.Ambito;
4 import com.onpe.presidenciales.model.Resultado;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8
9 public interface ResultadoRepository extends JpaRepository<Resultado, Long> {
10     List<Resultado> findByAmbito(Ambito ambito); no usages
11 }
```

Ahora seguimos con la creación del DTO, que sirve para enviar al frontend solo los datos necesarios de un resultado (nombre, partido, votos, porcentaje), sin exponer directamente la entidad completa de la base de datos.

```
Ambito.java x ResultadoDTO.java x
1 package com.onpe.presidenciales.dto;
2
3 public record ResultadoDTO( no usages
4     String nombre, no usages
5     String partido, no usages
6     Long votos, no usages
7     Double porcentaje no usages
8 ) {}
```

Continuamos con la creación del Service, que sirve para contener la lógica de negocio. En este caso para obtener los resultados filtrados por ámbito tanto Nacional o Extranjero y transformarlos de entidad a DTO antes de entregarlos.

```
data.sql ResultadoService.java x ResultadoRepository.java Resultado
1 package com.onpe.presidenciales.service;
2
3 import com.onpe.presidenciales.dto.ResultadoDTO;
4 import com.onpe.presidenciales.model.Ambito;
5 import com.onpe.presidenciales.model.Candidato;
6 import com.onpe.presidenciales.model.Resultado;
7 import com.onpe.presidenciales.repository.ResultadoRepository;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 import java.util.List;
12 import java.util.Map;
13 import java.util.stream.Collectors;
```

```
data.sql  ResultadoService.java x ResultadoRepository.java ResultadoController.java
14
15 @Service
16 public class ResultadoService {
17
18     @Autowired
19     private ResultadoRepository resultadoRepository;
20
21     public List<ResultadoDTO> obtenerPorAmbito(Ambito ambito) { 1 usage
22         return resultadoRepository.findByAmbito(ambito).stream() Stream<Resultado>
23             .map( Resultado r -> new ResultadoDTO(
24                 r.getCandidato().getNombre(),
25                 r.getCandidato().getPartido(),
26                 r.getVotos(),
27                 r.getPorcentaje()
28             )) Stream<ResultadoDTO>
29             .toList();
30     }
```

```
public List<ResultadoDTO> obtenerTotal() { no usages 1 related problem
    List<Resultado> todos = resultadoRepository.findAll();

    Map<Candidato, Long> votosPorCandidato = todos.stream()
        .collect(Collectors.groupingBy(
            Resultado::getCandidato,
            Collectors.summingLong(Resultado::getVotos)
        ));

    long totalGeneral = votosPorCandidato.values().stream() Stream<Long>
        .mapToLong(Long::longValue) LongStream
        .sum();

    return votosPorCandidato.entrySet().stream() Stream<Entry<...>>
        .map( Entry<Candidato, Long> e -> new ResultadoDTO(
            e.getKey().getNombre(),
            e.getKey().getPartido(),
            e.getValue(),
            porcentaje: (e.getValue() * 100.0) / totalGeneral
        )) Stream<ResultadoDTO>
        .toList();
}
```

Creación del controller para exponer el endpoint.

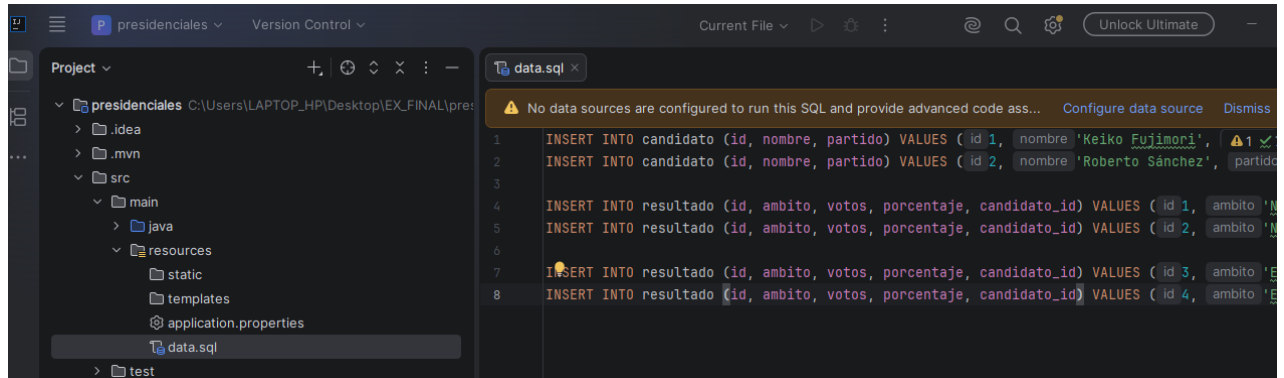
```
data.sql  ResultadoService.java  ResultadoRepository.java  ResultadoController.java x
1 package com.onpe.presidenciales.controller;
2
3 import com.onpe.presidenciales.dto.ResultadoDTO;
4 import com.onpe.presidenciales.model.Ambito;
5 import com.onpe.presidenciales.service.ResultadoService;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RequestParam;
10 import org.springframework.web.bind.annotation.RestController;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/api/resultados")
16 public class ResultadoController {
17
18     @Autowired
19     private ResultadoService resultadoService;
20
21     @GetMapping
22     public List<ResultadoDTO> obtenerResultados(@RequestParam Ambito ambito) {
23         return resultadoService.obtenerPorAmbito(ambito);
24     }
25
26     @GetMapping("/total")
27     public List<ResultadoDTO> obtenerTotal() {
28         return resultadoService.obtenerTotal();
29     }
30 }
```

Bien, ahora continuamos con la configuración de “application.properties”, que sirve para decirle a Spring Boot cómo conectarse a la base de datos. Pero en este caso he usado el H2 solo para probar que funciona.

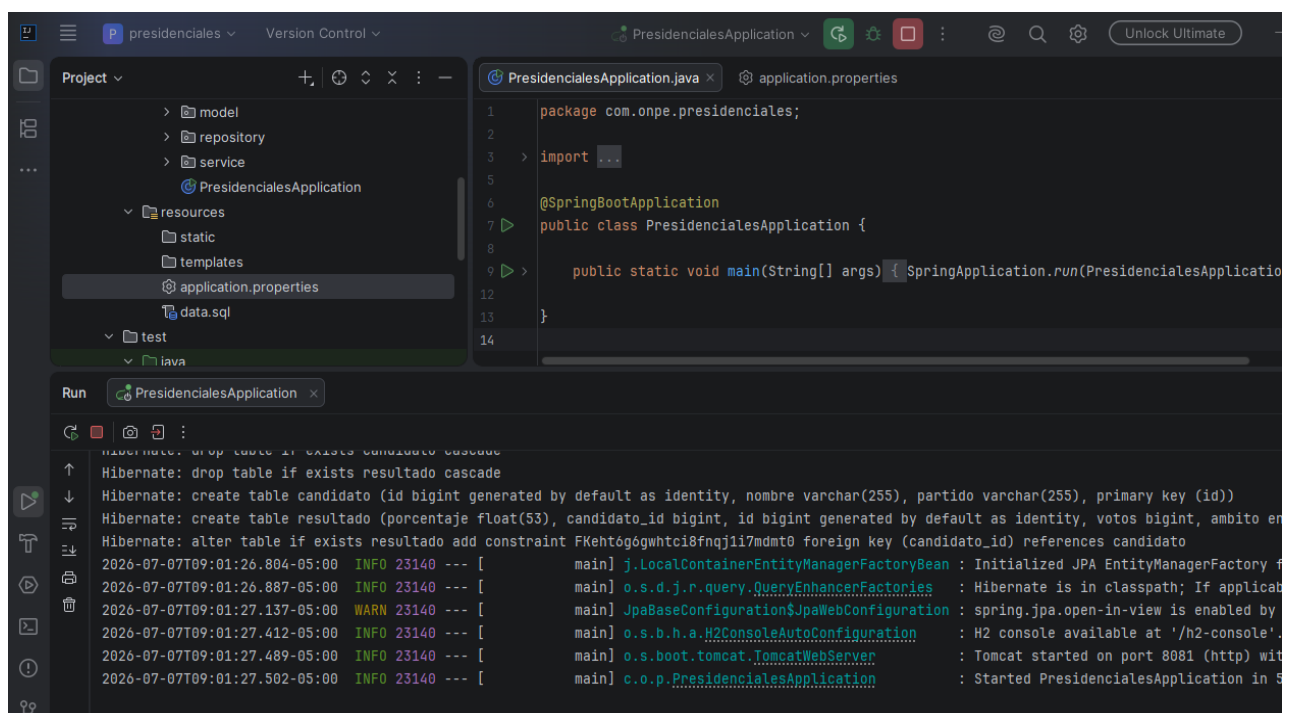
```
presidenciales  Version Control  Current File  Unlock Ultimate
Project  C:\Users\LAPTOP_HP\Desktop\EX_FINAL\presidenciales
  > .idea
  > .mvn
  > src
    > main
      > java
      > resources
        > static
        > templates
        application.properties
    > test

ResultadoDTO.java  ResultadoService.java  ResultadoController.java  application.properties x
1 spring.datasource.url=jdbc:h2:mem:onpe
2 spring.datasource.driverClassName=org.h2.Driver
3 spring.datasource.username=sa
4 spring.datasource.password=
5
6 spring.h2.console.enabled=true
7
8 spring.jpa.hibernate.ddl-auto=create-drop
9 spring.jpa.show-sql=true
10 spring.application.name=presidenciales
```

Ahora seguimos con la inserción de datos de prueba.

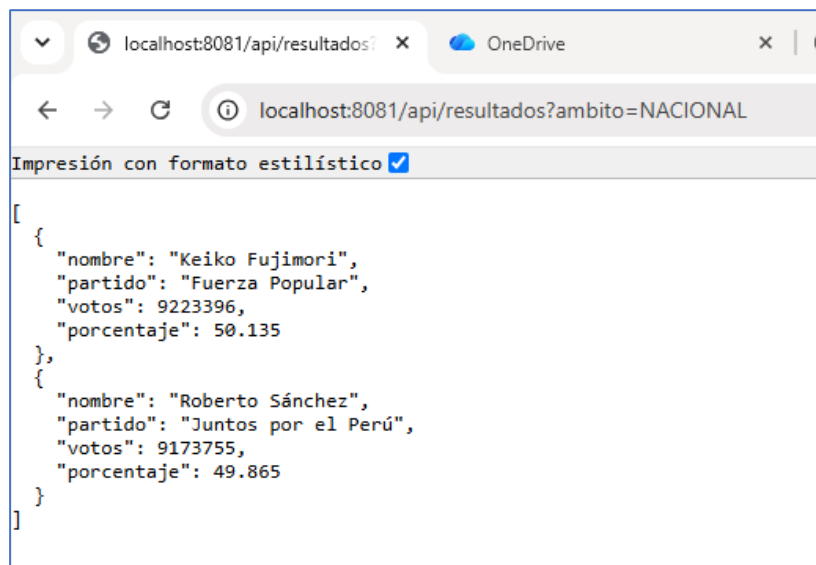


Corremos la aplicación:

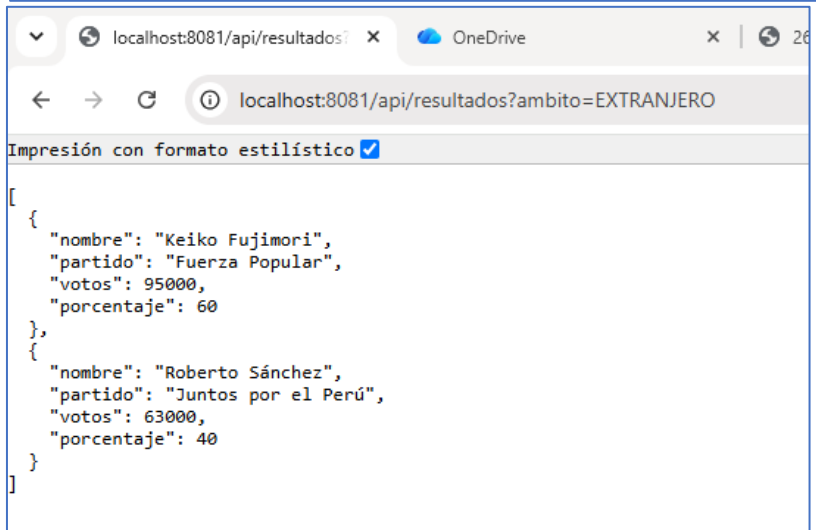


Cambié de puerto porque por alguna razón el 8080 no me abrió, así que cambié al 8081.

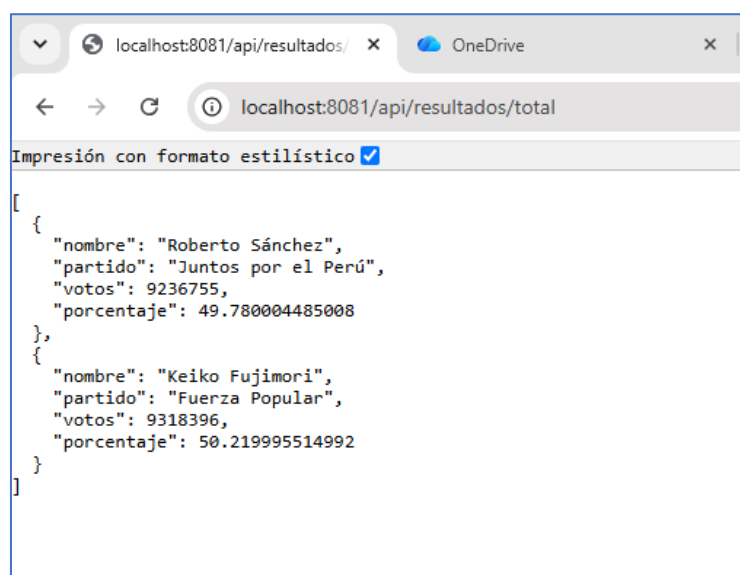
Y como resultado acá tenemos lo que se nos pide:



```
[
  {
    "nombre": "Keiko Fujimori",
    "partido": "Fuerza Popular",
    "votos": 9223396,
    "porcentaje": 50.135
  },
  {
    "nombre": "Roberto Sánchez",
    "partido": "Juntos por el Perú",
    "votos": 9173755,
    "porcentaje": 49.865
  }
]
```



```
[
  {
    "nombre": "Keiko Fujimori",
    "partido": "Fuerza Popular",
    "votos": 95000,
    "porcentaje": 60
  },
  {
    "nombre": "Roberto Sánchez",
    "partido": "Juntos por el Perú",
    "votos": 63000,
    "porcentaje": 40
  }
]
```



```
[
  {
    "nombre": "Roberto Sánchez",
    "partido": "Juntos por el Perú",
    "votos": 9236755,
    "porcentaje": 49.780004485008
  },
  {
    "nombre": "Keiko Fujimori",
    "partido": "Fuerza Popular",
    "votos": 9318396,
    "porcentaje": 50.219995514992
  }
]
```

Aquí se ven el resultado de las APIS que se piden en el inciso.

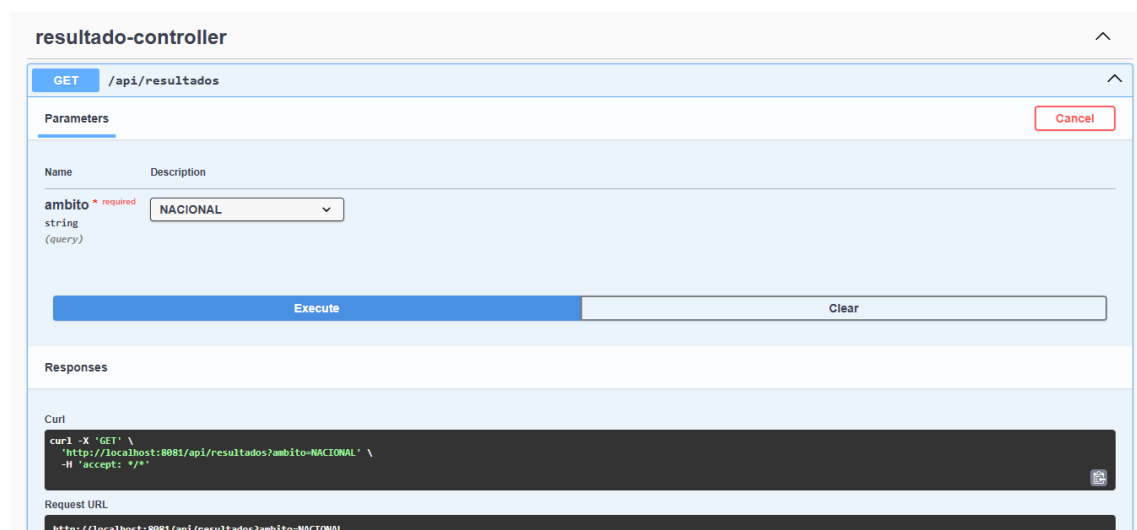
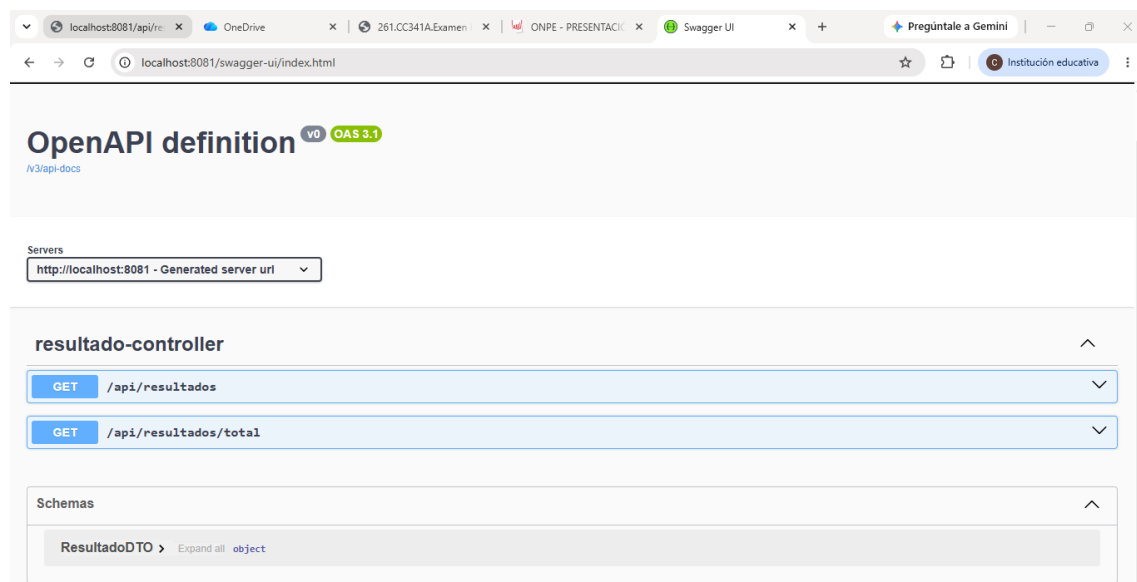
b) Demostración de funcionamiento de las apis con swagger u otro equivalente (1pt)

Aquí, vamos con la configuración de Swagger, springdoc-openapi, para la documentación automática de las APIs.

Entonces le he agregado esto al pom.xml

```
66         <dependency>
67             <groupId>org.springdoc</groupId>
68             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
69             <version>2.8.5</version>
70         </dependency>
71     </dependencies>
```

Abrimos swagger-ui



Code

Details

200

Response body

```
[
  {
    "nombre": "Keiko Fujimori",
    "partido": "Fuerza Popular",
    "votos": 9223396,
    "porcentaje": 50.135
  },
  {
    "nombre": "Roberto Sánchez",
    "partido": "Juntos por el Perú",
    "votos": 9173755,
    "porcentaje": 49.865
  }
]
```

 Download

Response headers

```
connection: keep-alive
content-length: 190
content-type: application/json
date: Tue, 07 Jul 2026 14:16:17 GMT
keep-alive: timeout=60
```

GET

/api/resultados/total

^

Parameters

Try it out

No parameters

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/resultados/total' \
  -H 'accept: */*'
```



Request URL

```
http://localhost:8081/api/resultados/total
```

Server response

Code

Details

200

Response body

```
[
  {
    "nombre": "Roberto Sánchez",
    "partido": "Juntos por el Perú",
    "votos": 9236755,
    "porcentaje": 49.780004485007964
  },
  {
    "nombre": "Keiko Fujimori",
    "partido": "Fuerza Popular",
    "votos": 9318396,
    "porcentaje": 50.219995514992036
  }
]
```

 Download

Response headers

```
connection: keep-alive
content-length: 214
content-type: application/json
date: Tue, 07 Jul 2026 14:17:30 GMT
keep-alive: timeout=60
```

Responses

Code

Description

Links

200

OK

No links

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "nombre": "string",
    "partido": "string",
    "votos": 9007199254740991,
    "porcentaje": 0.1
  }
]
```

Schemas

ResultadoDTO ^ Collapse all object

nombre string

partido string

votos integer int64

porcentaje number double

c) Implementación de la funcionalidad a nivel frontend (3.5pts)

d) uso del framework de estilos Bootstrap en el frontend (1pt)

e) Implementación de la base de datos que atienda la HU (1pt)